THE STATE UNIVERSITY OF ZANZIBAR (SUZA)

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

IMPLEMENTATAION REPORT

Project Name: Citizen Alert System

Version:1.0

Date: February 2026

Author(s):

Zahor Khalfan Omar (24BCS018)

Khadija Ali Said (24BCS026)

Abdulkadir Bakar Mbwana (24BCS028)

Status: Draft

ACADEMIC YEAR 2025/2026

Introduction

We built a Citizen Alert System using C language. The system allows government offices to send targeted alerts to citizens based on their region, status, and education level.

Technologies Used

Programming Language

C Language - Core system development

Data Structures

Linked Lists - Store office accounts, citizens, and notifications

Queue - Manage alerts in order

Development Tools

GCC Compiler: Compile C code into working program

VS Code: Write and edit code

Git/GitHub: Save and share code with team

Terminal: Run and test the program

System Modules

Module 1: Data Structures

- Citizen struct (stores id, name, region, status, education)
- OfficeAccount struct (stores id, username, password, office name, role)
- NotifNode struct (stores notification details)
- Queue struct (manages notifications in order)

Module 2: User Registration

- `citizen_register()` - Creates new citizen account
- Auto-generates Citizen ID starting from 100
- Stores citizen in linked list

Module 3: Office Login and Authentication

- `office_login()` - Verifies username and password
- `find_office_by_username()` - Searches for account
- Role-based access (Super Admin vs Admin)

Module 4: Notification Management

- `notif_create()` - Creates new notification
- `notif_list()` - Displays all notifications
- `notif_search()` - Finds notification by ID
- `notif_deactivate()` - Removes notification

Module 5: Priority Sorting

- `sort_by_priority()` - Sorts notifications (Emergency > High > Normal > Low)
- Uses bubble sort algorithm

Module 6: Citizen Alert Viewing

- `view_my_alerts()` - Shows alerts matching citizen profile
- `matches()` - Checks if alert matches citizen's region, status, or education

Module 7: Super Admin Functions

- sa_create_account() - Creates office accounts
- sa_list_accounts() - Shows all accounts
- sa_toggle_account() - Activates/deactivates accounts
- sa_delete_account() - Removes accounts
- sa_reset_password() - Changes passwords

Module 8: Helper Functions

- get_time() - Gets current timestamp
- clear_stdin() - Clears input buffer
- prio_str() - Converts priority number to text
- status_str() - Converts status number to text
- edu_str() - Converts education number to text

Data Flow

Step 1: User starts the program
Step 2: User chooses Office Admin or Citizen
Step 3: If Office Admin, login with username and password
Step 4: Super Admin can create ministry accounts
Step 5: Admin creates notifications with priority and targeting
Step 6: Citizens register with their information
Step 7: Citizens view alerts filtered by their profile
Step 8: Emergency alerts reach everyone

5. Key Functions Implementation

Function 1: Citizen Registration

```
static void citizen_register(void) {

    Citizen *c = malloc(sizeof(Citizen));

    c->id = cid_counter++;

}
```

Function 2: Create Notification

```
static void notif_create(OfficeAccount *acc) {

    NotifNode n;

    n.id = nid_counter++;

    get_time(n.created_at, MAX_STR);

    strncpy(n.office, acc->office_name, MAX_STR-1);

    // Get title, message, priority, target from user

    enqueue(n);

}
```

Function 3: View Alerts with Filtering

```
static int matches(Citizen *c, NotifNode *n) {
    if (n->priority == PRIO_EMERGENCY) return 1;
    switch (n->target_type) {
        case TGT_ALL: return 1;
        case TGT_REGION: return strstr(c->region, n->target_value) ? 1 : 0;
        case TGT_STATUS: return c->status == atoi(n->target_value);
        case TGT_EDUCATION: return c->education >= atoi(n->target_value);
    }
}
```

Testing Implementation

What we tested:

- Citizen registration works and assigns unique ID
- Office login works with correct credentials
- Wrong password is rejected
- Admin can create notifications
- Notifications are sorted by priority
- Citizens see only matching alerts
- Emergency alerts reach everyone
- Super Admin can manage accounts
- Regular Admin cannot delete other office notifications

Challenges During Implementation

Challenge 1: Sorting notifications by priority

- **Solution:** Created bubble sort function that runs before displaying

Challenge 2: Permission control for different roles

- **Solution:** Added role checking in every function

Challenge 3: Input validation to prevent crashes

- **Solution:** Created clear_stdin() function and added checks

Limitations of Current Implementation

- No file saving - data lost when program closes
- No password encryption - stored as plain text
- No network connectivity - offline only
- No session timeout - user must logout manually
- English only - no Swahili support

Conclusion

The Citizen Alert System was successfully implemented using C language. All functional requirements were met including citizen registration, office login, notification creation with targeting, priority sorting, and role-based access control. The system allows government ministries to send targeted alerts and citizens to receive only relevant information. Emergency alerts successfully reach all citizens regardless of their profile.